

Institución Universitaria Pascual Bravo

Inteligencia de Negocios · Grupo 50 · 2026-II

TRABAJO DE CONSULTA

SolarBI Pascual

Power BI y Grafana, gobernanza de datos, automatización ETL e IoT

Integrante 1: Samuel Alfonso Avendaño

Integrante 2: [PENDIENTE: nombre completo integrante 2]

Docente: Ramiro Grisales Montoya

7 de octubre de 2026

PARTE A — Consulta teórica

PARTE A — Consulta teórica (19 preguntas)

Cada respuesta ≤ 120 palabras (las tablas, el diagrama y el código no cuentan en ese límite), con fuente propia y aplicada a SolarBI Pascual.

BLOQUE 1 — Power BI y Grafana

D1. ¿Qué es Grafana?

Grafana es una plataforma open-source de observabilidad y visualización que consulta datos en vivo desde *data sources* (bases de datos, series temporales, APIs) sin copiarlos a un modelo propio. Un *panel* es una visualización individual (gráfico, tabla, stat); un *dashboard* agrupa paneles relacionados; las *variables* parametrizan consultas (p. ej. elegir un dispositivo) y el *alerting* evalúa reglas sobre los datos y dispara notificaciones cuando se cruzan umbrales. **Grafana OSS** se instala y administra uno mismo (gratis, sin límites artificiales); **Grafana Cloud** es el mismo motor como SaaS gestionado, con planes pagos y retención administrada. En SolarBI Pascual se usa Grafana OSS sobre PostgreSQL para monitoreo casi en tiempo real de irradiancia, potencia y % de datos válidos del inversor.

Fuente: Grafana Labs, “What is Grafana?”, <https://grafana.com/docs/grafana/latest/introduction/>

D2. Power BI vs. Grafana — tabla comparativa

Criterio	Power BI	Grafana (OSS)
Propósito principal	BI empresarial: reportes y modelos de negocio	Observabilidad y monitoreo operativo casi en tiempo real
Usuario típico	Analista de negocio, directivo	Ingeniero/operador de planta
Conexión a datos	Import o DirectQuery a decenas de conectores	Consulta en vivo a data sources (SQL, series temporales)
Modelo semántico	Sí, modelo tabular con relaciones	No; consulta directa al origen por panel
Lenguaje	DAX (medidas) + Power Query (M)	SQL/PromQL/Flux según data source, sin modelo propio
DAX vs. consulta del origen	Cálculos centralizados en medidas DAX	Cálculo delegado a la consulta de cada panel
Actualización	Scheduled Refresh (mín. cada hora en capacidad compartida)	Auto-refresh configurable en segundos
Tiempo real	Limitado (DirectQuery/streaming dataset)	Nativo, pensado para eso
Alertas	Data alerts básicas en Service	Alerting robusto, multicanal
Licenciamiento	Propietario (Microsoft)	Open-source (AGPL)
Costo	Pro/Premium por usuario o capacidad	Gratis (OSS); Cloud es pago
Control de acceso	RLS, Azure AD, workspaces	Org/Team/Folder permissions

Para SolarBI: Power BI conviene para el reporte mensual de yield y ahorro hacia el directivo; Grafana conviene para que el operador vigile irradiancia y potencia casi en vivo.

Fuente: Microsoft Learn, “What is Power BI”, <https://learn.microsoft.com/power-bi/fundamentals/power-bi-overview>; Grafana Labs Docs.

D3. Dashboards operativo/analítico/estratégico y KPIs de SolarBI

Un dashboard **operativo** muestra el estado casi en tiempo real para reaccionar ya (ej. alarmas); uno **analítico** permite explorar tendencias y comparar periodos; uno **estratégico** resume indicadores de alto nivel para decisiones de negocio en horizontes largos.

KPI	Tipo de dashboard	Herramienta	Justificación
Energía	Analítico	Power BI	Serie histórica para comparar periodos
Yield	Estratégico	Power BI	Resume desempeño de la planta para directivos
Performance Ratio	Analítico	Power BI	Requiere comparar generado vs. esperado en el tiempo
Ahorro	Estratégico	Power BI	Traduce desempeño técnico a valor de negocio (COP)
Disponibilidad	Operativo	Grafana	El operador necesita saber ya si un equipo está caído
Alarmas	Operativo	Grafana	Exige alerting y reacción inmediata
% Datos válidos	Operativo/Analítico	Grafana	Vigilancia continua de calidad de la ingesta

Fuente: Microsoft Learn, “Dashboards in Power BI”, <https://learn.microsoft.com/power-bi/create-reports/service-dashboards>.

D4. Import, DirectQuery, Scheduled Refresh vs. auto-refresh de Grafana

Import carga una copia del dato al modelo tabular en memoria: consultas rapidísimas, pero el dato queda tan actualizado como la última **Scheduled Refresh** (programable, mínimo horario en capacidad compartida). **DirectQuery** consulta el origen en cada interacción del usuario, sin copiar datos, pero es más lenta y depende del rendimiento de la fuente. Para SolarBI, con solo 288 lecturas/día por dispositivo ya agregadas a `dwh.fact_energia_dia`, **Import** es la opción correcta: el volumen es pequeño y el refresco diario (tras correr el ETL) es suficiente para yield/ahorro. Esto contrasta con el **auto-refresh** de Grafana, que reconsulta PostgreSQL cada pocos segundos/minutos sin copiar datos — apropiado para vigilar irradiancia y potencia casi en vivo, no para reportes de negocio.

Fuente: Microsoft Learn, “Data sources for Power BI”, <https://learn.microsoft.com/power-bi/connect-data/power-bi-data-sources>.

D5. Variables/templating (Grafana) vs. slicers/filtros (Power BI)

Ambos mecanismos parametrizan una visualización sin duplicar paneles. Una **variable de Grafana** se declara una vez en el dashboard y se inserta dentro del *raw SQL* de cada panel (p. ej. `$dispositivo`), reescribiendo literalmente la consulta enviada a PostgreSQL. Un **slicer/filtro de Power BI** no reescribe DAX: aplica un contexto de filtro sobre el modelo tabular ya cargado en memoria, y las medidas DAX se recalculan respetando ese contexto. Para SolarBI se propone la variable `dispositivo` en Grafana (`SELECT DISTINCT dispositivo_id FROM silver.lectura_5min`); al seleccionarla, el panel de series de tiempo sustituye `WHERE dispositivo_id = '$dispositivo'` en su consulta SQL antes de ejecutarla, mostrando solo las lecturas de ese inversor.

Fuente: Grafana Labs Docs, “Templates and variables”, <https://grafana.com/docs/grafana/latest/dashboards/variables/>.

BLOQUE 2 — Gobernanza de datos

G1. Data Governance vs. Data Management vs. Data Quality (DAMA-DMBOK)

Según DAMA-DMBOK, **Data Management** es la disciplina completa de planear, ejecutar y supervisar el ciclo de vida del dato (incluye ETL, modelado, seguridad, calidad). **Data Governance** es el componente que define **quién decide** sobre los datos: políticas, roles y responsabilidades (ej. quién aprueba cambios en `fact_energia_dia`). **Data Quality** es una de las áreas de conocimiento dentro de Data Management: medir y mejorar exactitud, completitud y consistencia (en SolarBI, el porcentaje_datos_validos calculado por el ETL). En el proyecto: Governance decide que el Data Owner autorice cambios de esquema del simulador; Management ejecuta el pipeline Bronze-Silver-Gold; Quality aplica las 3 reglas de validación y reporta sus métricas.

Fuente: DAMA International, *DAMA-DMBOK: Data Management Body of Knowledge*, 2.^a edición, cap. 1 y 3.

G2. Roles Data Owner, Steward, Custodian y su relación con el equipo

El **Data Owner** es responsable del negocio sobre un dominio de datos (decide qué significa “válido”); el **Data Steward** traduce esa decisión en reglas operativas y vigila la calidad día a día; el **Data Custodian** administra la infraestructura técnica donde vive el dato (PostgreSQL).

Rol DAMA	Rol del equipo SolarBI
Data Owner	Product Owner (define qué KPI importa)
Data Steward	BI Analyst (valida reglas de calidad y KPIs)
Data Custodian	Data Engineer (administra PostgreSQL, ETL)
—	Data Modeler (diseña <code>fact_energia_dia</code> , modelo DAX)

En un equipo de dos personas estos roles se combinan, pero conviene nombrarlos explícitamente para que quede claro quién aprueba cambios.

Fuente: DAMA International, *DAMA-DMBOK*, cap. 3, “Data Governance”.

G3. Data Contract aplicado a SolarBI

Un *data contract* es un acuerdo formal entre quien produce un dato y quien lo consume, y debe incluir como mínimo: **esquema** (nombres y tipos de columnas), **unidades** (kW, W/m², °C), **frecuencia** (cada 5 min), **responsable** (Data Owner/Steward) y **niveles de calidad** (umbral mínimo de % datos válidos). Si la fuente cambia un campo sin avisar (p. ej. renombra `p_ac_kw` o cambia sus unidades a W), Power BI fallará el *Scheduled Refresh* o cargará valores mal escalados sin error visible hasta que el Yield se vea absurdo; en Grafana, el panel puede quedar en blanco o mostrar ceros silenciosamente. Para SolarBI se propone fijar el contrato en `sql/tables.sql` y documentarlo en el README, versionado junto al ETL.

Fuente: DAMA International, *DAMA-DMBOK*, cap. 3 y 11 (interoperabilidad y metadatos).

G4. Single Source of Truth, catálogo de KPIs y riesgo de duplicar cálculo

El **Single Source of Truth (SSOT)** es el lugar único y autorizado donde se calcula un dato; en SolarBI es `dwh.fact_energia_dia` en PostgreSQL. Un **catálogo de KPIs** documenta, para cada indicador, su definición exacta.

Campo	Yield	% Datos Válidos
Nombre	Yield (kWh/kWp)	% de datos válidos
Fórmula	<code>energía_kwh / potencia_nominal_kWp</code>	<code>filas_validas / filas_leidas × 100</code>
Unidad	kWh/kWp	%

Fuente	dwh.fact_energia_dia	silver.metricas_calidad
Responsable	BI Analyst	Data Engineer
Frecuencia	Diaria	Por corrida del ETL

Si Yield se calcula en DAX y por separado en SQL/Grafana, ambos pueden divergir por redondeos o por usar distinta potencia nominal, rompiendo el SSOT y generando desconfianza en el tablero.

Fuente: DAMA International, *DAMA-DMBOK*, cap. 12, “KPI and metric governance”.

G5. Ley 1581 de 2012 aplicada a SolarBI

La Ley 1581 define dato personal como información vinculada o asociable a una persona natural determinada o determinable. Las lecturas de `p_ac_kw`, `irradiancia_wm2` o `temp_modulo_c` **no son datos personales por sí solas**: describen un equipo, no a una persona. Esto cambia si el tablero muestra **qué operador atendió cada alarma**: ahí el dato técnico queda vinculado a un usuario identificable y pasa a ser dato personal sujeto a la Ley (finalidad, acceso restringido, posible anonimización en reportes). El equipo debe separar siempre telemetría técnica de registros de atención por operador, aplicar control de acceso a este último y no asumir que todo dato de IoT es personal, ni lo contrario.

Fuente: Congreso de Colombia, Ley 1581 de 2012, art. 3, http://www.secretariasenado.gov.co/senado/basedoc/ley_1581_2012.html.

BLOQUE 3 — Automatización ETL

E1. ETL vs. ELT, capas Bronze/Silver/Gold y Power Query

ETL transforma el dato *antes* de cargarlo al destino (como hace `run_etl.py`: limpia en Python y luego inserta en PostgreSQL). **ELT** carga primero el dato crudo y transforma *dentro* del motor destino (ej. con SQL en PostgreSQL). Bronze es el dato crudo sin transformar (el CSV del simulador); Silver aplica limpieza/validación (reglas de calidad); Gold agrega para consumo de negocio (`fact_energia_dia`). **Power Query sí puede considerarse una herramienta ETL**: extrae de múltiples orígenes, aplica transformaciones declarativas (tipos, filtros, uniones) en su motor M, y carga al modelo de Power BI — cumple las tres fases, aunque limitada a transformaciones orientadas a análisis, no a cargas incrementales complejas como un ETL de producción.

Fuente: Microsoft Learn, “Power Query what is”, <https://learn.microsoft.com/power-query/power-query-what-is-power-query>.

E2. Full Load, Incremental Load e idempotencia en fact_energia_dia

Full Load reprocesa y recarga todo el histórico en cada corrida; es simple pero costoso a escala. **Incremental Load** procesa solo lo nuevo o cambiado desde la última corrida, más eficiente pero más complejo de controlar. **Idempotencia** significa que ejecutar el mismo proceso varias veces produce el mismo resultado final. En `fact_energia_dia`, la clave (`fecha_key`, `dispositivo_key`) identifica de forma única cada fila agregada; `run_etl.py` usa `INSERT ... ON CONFLICT (fecha_key, dispositivo_key) DO UPDATE (UPSERT)`, de modo que si el proceso corre dos veces sobre el mismo Bronze, la segunda corrida *actualiza* las mismas filas en vez de insertarlas de nuevo. Esto se verificó realmente: 3 filas antes y 3 filas después de la segunda ejecución (ver evidencia en Parte B).

Fuente: PostgreSQL Docs, “INSERT ... ON CONFLICT”, <https://www.postgresql.org/docs/current/sql-insert.html#SQL-ON-CONFLICT>.

E3. cron/Programador de tareas, Airflow y Scheduled Refresh

Opción	Qué es	Qué automatiza	Ventajas	Limitaciones	Cuándo conviene en SolarBI
cron / Task Scheduler	Programador del sistema operativo	Ejecutar un comando a una hora fija	Simple, sin instalar nada extra	Sin reintentos, dependencias ni monitoreo built-in	Equipo de 2 personas, un solo ETL diario
Apache Airflow	Orquestador de workflows (DAGs)	Dependencias entre tareas, reintentos, alertas	Observabilidad, escalable, maneja pipelines complejos	Sobra para un ETL de un solo script; curva de aprendizaje	Si SolarBI creciera a múltiples plantas/pipelines
Scheduled Refresh (Power BI)	Programador propio de Power BI Service	Refrescar el modelo tabular	Integrado, sin código	No ejecuta el ETL de PostgreSQL, solo el refresh del dataset	Complementa a cron: refresca el reporte después del ETL

Para el volumen y equipo de SolarBI, **cron** es la opción más razonable hoy.

E4. Incremental Refresh en Power BI

El Incremental Refresh particiona una tabla grande por fecha, usando los parámetros `RangeStart` y `RangeEnd` como filtro dinámico de Power Query: Power BI genera particiones (p. ej. mensuales) y solo refresca las particiones dentro de la “ventana de refresco” configurada, dejando las particiones históricas intactas. Esto reduce drásticamente el tiempo de Scheduled Refresh frente a recargar toda la tabla. Para SolarBI, con solo 3 días y 864 lecturas, **no se justifica** usar Incremental Refresh — el costo de configurarlo supera el beneficio; conviene solo si el histórico creciera a meses/años de lecturas de 5 minutos, donde recargar todo cada vez sería lento y costoso en la capacidad de Power BI Service.

Fuente: Microsoft Learn, “Incremental refresh for datasets”, <https://learn.microsoft.com/power-bi/connect-data/incremental-refresh-overview>.

BLOQUE 4 — Internet de las Cosas

I1. Arquitectura IoT por capas aplicada a SolarBI

```
[Inversor 5kWp] --(1) dispositivo
|
[Gateway/Edge] --(2) agrega/pre-procesa localmente
|
[Broker MQTT / Red] --(3) transporte pub/sub
|
[PostgreSQL] --(4) almacenamiento <-- aquí viven Silver y Gold
|
[Power BI / Grafana] --(5) aplicación/visualización
```

Capas Bronze/Silver/Gold respecto a esta arquitectura: **Bronze** es el dato tal como sale del inversor/gateway, antes o justo al llegar al almacenamiento (el CSV crudo en este proyecto simula esa llegada); **Silver** vive dentro de la capa de almacenamiento ya validado (`silver.lectura_5min`); **Gold** también está en almacenamiento pero agregado para consumo (`dwh.fact_energia_dia`), ya en la frontera con la capa de aplicación (Power BI/Grafana).

Fuente: Microsoft Learn, “IoT reference architecture”, <https://learn.microsoft.com/azure/architecture/reference-architectures/iot>.

I2. MQTT y jerarquía de topics para SolarBI

MQTT es un protocolo pub/sub liviano: un cliente **publica** mensajes a un **topic** en un **broker**, y otros clientes se **suscriben** a ese topic para recibirlos sin conexión directa entre sí. **QoS 0** entrega a lo sumo una vez (sin confirmación, puede perderse); **QoS 1** garantiza al menos una vez (puede duplicarse); **QoS 2** garantiza exactamente una vez (máxima fiabilidad, mayor costo). Jerarquía propuesta: `pascualbravo/solar/campus-robledo/inv-01/p_ac_kw`, `pascualbravo/solar/campus-robledo/inv-01/irradiancia_wm2`, `pascualbravo/solar/campus-robledo/inv-01/alarma`. Para telemetría continua (potencia, irradiancia) conviene **QoS 0 o 1**; para alarmas críticas, **QoS 2**, porque perder o duplicar una alarma tiene más costo que perder una lectura de rutina.

Fuente: MQTT.org / OASIS, *MQTT Version 5.0 Specification*, <https://mqtt.org/mqtt-specification/>.

I3. Modbus RTU/TCP vs. MQTT vs. OPC UA

Criterio	Modbus RTU/TCP	MQTT	OPC UA
Modelo de comunicación	Maestro-esclavo (polling)	Publish/Subscribe	Cliente-servidor + pub/sub
Uso típico	Campo: PLCs, inversores, medidores	Transporte IoT a la nube/broker	Interoperabilidad industrial estandarizada
Formato de datos	Binario, registros crudos	Payload libre (JSON/binario)	Modelado semántico tipado
Seguridad	Nula/mínima nativa (RTU/TCP clásico)	TLS + auth a nivel de broker	Seguridad nativa (cifrado, certificados)
Ventajas	Simple, muy extendido en campo	Liviano, ideal para redes inestables	Rico en semántica, seguro por diseño
Limitaciones	Sin seguridad, propietario por fabricante	No define semántica de los datos	Más pesado de implementar

Más probable en **inversores y medidores** de SolarBI: **Modbus TCP/RTU** en campo (protocolo nativo de la mayoría de inversores comerciales), con un gateway que traduce a **MQTT** hacia el broker/nube; OPC UA aparece más en integraciones industriales mayores, menos común en inversores residenciales.

Fuente: Modbus Organization, *Modbus Application Protocol Specification*, <https://modbus.org/specs.php>.

I4. Batch vs. Streaming aplicado a telemetría de 5 minutos

Batch procesa datos acumulados en lotes periódicos (el ETL de SolarBI lee todo el Bronze de una vez). **Streaming** procesa cada evento casi al instante de su llegada. La telemetría cada 5 minutos ya es un *batch* de baja frecuencia; al agregarla a `fact_energia_dia` (diario) se **gana** simplicidad de consulta y un tablero más liviano, pero se **pierde** granularidad para detectar anomalías puntuales (un pico de potencia negativa a las 14:03 desaparece en el total del día). Para SolarBI conviene **batch** en ambas granularidades: Grafana consulta `silver.lectura_5min` (5 min) y Power BI consulta `fact_energia_dia` (diario). Streaming real (MQTT con procesamiento continuo) solo se justificaría si SolarBI necesitara alertas en segundos, no minutos.

Fuente: Microsoft Learn, “Batch vs. real-time processing”, <https://learn.microsoft.com/azure/architecture/data-guide/big-data/batch-processing>.

BLOQUE 5 — Control de versiones

V1. Git, GitHub, .pbix vs. .pbip, y versionar Grafana con JSON

Git es el sistema de control de versiones distribuido (historial local); **GitHub** es una plataforma que aloja repositorios Git en la nube y agrega colaboración (PRs, issues). Un **repository** contiene el historial del proyecto; un **commit** es una fotografía versionada de cambios; una **branch** es una línea de desarrollo paralela; un **pull request** propone fusionar una branch con revisión. El **.pbix** es un binario comprimido (ZIP) que mezcla datos, modelo y reporte: Git no muestra diffs legibles ni fusiona cambios línea a línea. El

.pbip separa reporte y modelo en archivos de texto (JSON/TMDL), permitiendo diffs y revisiones reales en GitHub. Un dashboard de Grafana se versiona exportándolo como **JSON** (grafana/dashboard.json) y confirmándolo en un commit cada vez que cambia.

Fuente: Microsoft Learn, *"Power BI Projects (PBIP)"*, <https://learn.microsoft.com/power-bi/developer/projects/projects-overview> ; Git Docs, <https://git-scm.com/doc>.

PARTE B — Práctica

Paso 1 — Bronze

Procedimiento: se ejecutó `python3 etl/simulador.py`, que genera 3 días × 288 lecturas/día (864 lecturas base) de un inversor de 5 kWp, inyectando anomalías controladas (potencia negativa, irradiancia faltante, filas duplicadas). Salida real de la ejecución:

```
=== Simulador SolarBI Pascual – resumen de ejecución ===
lecturas_base: 864
potencia_negativa_inyectada: 6
irradiancia_faltante_inyectada: 8
duplicados_inyectados: 8
total_filas_bronze: 872
Archivo generado: data/bronze/lecturas_bronze.csv
```

Explicación: el archivo Bronze resultante (872 filas, incluyendo las anomalías inyectadas) se trata como crudo e inmutable a partir de este punto; ninguna etapa posterior lo modifica.

Paso 2 — Silver

Procedimiento: `etl/run_etl.py` leyó el Bronze y aplicó las 3 reglas de calidad. Salida real:

```
--- Métricas de calidad (Silver) ---
filas_leidas: 872
filas_rechazadas_por_rango: 5
filas_rechazadas_por_duplicados: 7
filas_rechazadas_por_faltantes: 8
filas_validas: 852
porcentaje_datos_validos: 97.71
```

Explicación: $\text{porcentaje_datos_validos} = \text{filas_validas} / \text{filas_leidas} \times 100 = 852 / 872 \times 100 = 97.71 \%$. Las 20 filas rechazadas se reparten en las tres reglas (rango físico, duplicados, faltantes), cada una contada una sola vez por fila.

Paso 3 — PostgreSQL / Gold

Procedimiento: se crearon los esquemas con `psql -f sql/tables.sql` y el ETL cargó `silver.lectura_5min` y agregó `dwh.fact_energia_dia` con UPSERT. Consulta real ejecutada contra PostgreSQL:

```
SELECT * FROM dwh.fact_energia_dia ORDER BY fecha_key;
```

fecha_key	dispositivo_key	energia_kwh	pct_datos_validos
2026-09-01	INV-01	29.299	97.71
2026-09-02	INV-01	29.479	97.71
2026-09-03	INV-01	28.715	97.71

Explicación: la energía diaria se calculó agregando $p_{ac_kw} \times 5/60$ por día y dispositivo sobre las 852 lecturas válidas de Silver; ~29 kWh/día en un inversor de 5 kWp corresponde a un Yield de ~5.8–5.9 kWh/kWp, un valor físicamente razonable para una planta solar.

Idempotencia (obligatorio)

Procedimiento: se ejecutó `run_etl.py` **dos veces seguidas** sobre el mismo Bronze. Salida real de ambas corridas:

```
>>> PRIMERA EJECUCIÓN
silver.lectura_5min: antes=0   -> después=852
dwh.fact_energia_dia: antes=0  -> después=3

>>> SEGUNDA EJECUCIÓN
silver.lectura_5min: antes=852 -> después=852
dwh.fact_energia_dia: antes=3   -> después=3
```

Explicación: los conteos se mantuvieron exactamente iguales en la segunda corrida (852 y 3), demostrando idempotencia real gracias al UPSERT sobre las claves (`ts`, `dispositivo_id`) y (`fecha_key`, `dispositivo_key`) — no es una afirmación sin evidencia, son conteos verificados en PostgreSQL.

Paso 4 — Power BI

Estado: no se ejecutó en este entorno (sin Power BI Desktop disponible en el contenedor Linux donde se desarrolló el proyecto). Se documentó el código propuesto, completo y ejecutable contra los nombres reales de tabla, en `powerbi/medidas_dax.md`: conexión PostgreSQL en modo Import, parámetro de tarifa y las medidas DAX Energía Total, Yield, Ahorro Estimado y Porcentaje Datos Validos.

Resultado esperado: con los datos reales ya cargados (ver Paso 3), la tarjeta de Yield debería mostrar ~5.8–5.9 kWh/kWp y el gráfico de energía diaria replicar los 3 valores de `fact_energia_dia` de arriba.

Captura que debe tomar el estudiante: la página completa de Power BI Desktop (3 tarjetas + gráfico de columnas) conectada a la base solarbi real, generada siguiendo powerbi/medidas_dax.md.

Paso 5 — Grafana

Estado: Grafana OSS no se pudo instalar en este entorno (descarga y apt-get bloqueados por la política de red del contenedor; sin daemon Docker disponible). Detalle completo del intento y el error real en docs/ruta_alternativa_grafana.md. Se entrega el dashboard propuesto, listo para importar, en grafana/dashboard.json (panel de series de tiempo con \$__timeFilter, panel de % de datos válidos con umbrales, y variable dispositivo).

Captura que debe tomar el estudiante: al importar dashboard.json en una instancia real de Grafana OSS conectada a la misma base solarbi, tomar captura de ambos paneles con datos reales.

Paso 6 — Reflexión

Ver docs/reflexion_final.md (máx. 10 líneas), incluida íntegra:

Reflexión final (Parte B, paso 6) — máx. 10 líneas

Power BI resolvió mejor el reporte de negocio: medidas DAX centralizadas (Yield, Ahorro) dan una sola cifra confiable y fácil de presentar. Grafana resolvió mejor la vigilancia casi en tiempo real, con \$__timeFilter y variables de dashboard listas para explorar cualquier dispositivo sin tocar consultas. Lo más difícil fue demostrar idempotencia real con UPSERT y PostgreSQL corriendo en el mismo entorno que el ETL, y no poder instalar Grafana OSS en el entorno de desarrollo por restricciones de red (ver ruta alternativa documentada). Para el **operador de planta** usáramos **Grafana**, por su refresco rápido y alertas. Para un **directivo** usáramos **Power BI**, porque traduce la operación a KPIs de negocio (ahorro en COP, yield) en un reporte pulido y fácil de compartir.

PARTE C — GitHub

Repositorio

Nombre propuesto: solarbi-avendano-[PENDIENTE] (público). Un integrante lo crea y agrega al otro como colaborador; el colaborador debe aceptar la invitación antes de entregar. **Enlace:** [PENDIENTE: URL del repositorio en GitHub].

Colaboradores

Captura pendiente: pantalla de *Settings* → *Collaborators* mostrando a los dos integrantes, a tomar una vez creado el repositorio real en GitHub.

Commits

Mínimo 3 commits descriptivos por persona — propuesta completa en docs/commits_propuestos.md. **Captura pendiente:** historial de commits (git log o la vista de GitHub) mostrando ambos autores, una vez realizados los commits reales.

Estructura

```
solarbi-apellido1-apellido2/
├── README.md
├── .gitignore
├── .env.example
├── requirements.txt
├── docs/
│   ├── ruta_alternativa_grafana.md
│   ├── cron.md
│   ├── commits_propuestos.md
│   └── PDF de esta consulta
├── data/
│   ├── bronze/lecturas_bronze.csv
│   └── silver/lecturas_silver.csv
├── etl/
│   ├── simulador.py
│   └── run_etl.py
├── sql/
│   ├── tables.sql
│   └── cargas.sql
├── powerbi/
│   └── medidas_dax.md
└── grafana/
    └── dashboard.json
```

README

Ver README.md completo en el repositorio (nombres, curso, grupo, descripción, objetivo, arquitectura, tecnologías, instalación, configuración, ejecución, estructura, proceso ETL, reglas de calidad, PostgreSQL, Power BI, Grafana, reproducción y tabla de distribución del trabajo).

REFERENCIAS

1. Microsoft Learn. *What is Power BI*. <https://learn.microsoft.com/power-bi/fundamentals/power-bi-overview>
2. Microsoft Learn. *Incremental refresh for datasets*. <https://learn.microsoft.com/power-bi/connect-data/incremental-refresh-overview>
3. Microsoft Learn. *Power BI Projects (PBIP)*. <https://learn.microsoft.com/power-bi/developer/projects/projects-overview>
4. Grafana Labs. *Grafana documentation*. <https://grafana.com/docs/grafana/latest/>
5. PostgreSQL Global Development Group. *INSERT ... ON CONFLICT*. <https://www.postgresql.org/docs/current/sql-insert.html>
6. DAMA International. *DAMA-DMBOK: Data Management Body of Knowledge*, 2.^a ed.
7. Congreso de Colombia. *Ley 1581 de 2012*. http://www.secretariasenado.gov.co/senado/basedoc/ley_1581_2012.html
8. MQTT.org / OASIS. *MQTT Version 5.0 Specification*. <https://mqtt.org/mqtt-specification/>
9. Modbus Organization. *Modbus Application Protocol Specification*. <https://modbus.org/specs.php>
10. Git. *Git Documentation*. <https://git-scm.com/doc>